Prompted Segmentation for Drywall Quality Assurance

Text-Conditioned Segmentation using CLIPSeg

Crack Detection	Taping Detection	Inference Speed
0.47	0.61	45ms
mIoU	mIoU	per image

A single model that segments both **cracks** and **taping areas** from natural language prompts, trained on 4,400+ images with proper polygon annotations.

Mohith

March 2026

Contents

1 Executive Summary **2**

2 Model Selection & Rationale **2**
2.1 Why Text-Conditioned Segmentation? . . . 2
2.2 Model Comparison . . . 2

3 Dataset Preparation **3**
3.1 Data Sources . . . 3
3.2 Challenge: Annotation Format Mismatch . . . 3
3.3 Solution: Dataset Conversion Pipeline . . . 3
3.4 Final Dataset Statistics . . . 3

4 Training Configuration **3**
4.1 Architecture . . . 4
4.2 Hyperparameters . . . 4
4.3 Loss Function . . . 4
4.4 Prompt Augmentation . . . 4

5 Results **5**
5.1 Training Progress . . . 5
5.2 Test Set Performance . . . 5
5.3 Prompt Consistency . . . 6
5.4 Visual Results . . . 7
5.5 Runtime & Footprint . . . 8

6 Failure Analysis & Future Directions **9**
6.1 Observed Failure Modes . . . 9
6.2 Primary Bottleneck: Decoder Resolution . . . 9
6.3 Recommended Architecture Improvements . . . 10

7 Conclusion **10**

1 Executive Summary

Objective: Build a text-conditioned segmentation model that produces binary masks for "`segment crack`" and "`segment taping area`" prompts.

Approach: Fine-tuned CLIPSeg (decoder-only) on 4,473 training images with prompt augmentation.

Model	CLIPSeg (CIDAS/clipseg-rd64-refined)
Training Time	1 hour 4 minutes (25 epochs)
Trainable Parameters	1.78M / 150.7M (1.2%)
Test mIoU (Crack)	0.4723
Test mIoU (Taping)	0.6069
Inference Speed	45.4 ± 0.9 ms/image

2 Model Selection & Rationale

2.1 Why Text-Conditioned Segmentation?

Traditional segmentation requires separate models or output heads per class. Text-conditioned models offer:

- **Single model** handles multiple segmentation tasks
- **Prompt flexibility** — responds to semantic variations
- **Zero-shot potential** — can generalize to new defect types

2.2 Model Comparison

Table 1: Evaluated approaches for text-conditioned segmentation

Model	Advantages	Disadvantages	Speed
CLIPSeg (Selected)	Native text prompts, lightweight, easy fine-tuning	64×64 decoder resolution	45ms
SAM + CLIP	High-res masks, state-of-the-art	Complex pipeline, heavy	200ms
Grounded-SAM	Best quality	Requires 24GB+ VRAM	500ms

Selection Rationale

CLIPSeg was chosen for its **native text conditioning**, **efficient fine-tuning** (only decoder trained), and **production-ready inference speed**. The 64×64 output resolution is acceptable for this QA application where region detection matters more than pixel-perfect boundaries.

3 Dataset Preparation

3.1 Data Sources

Dataset	Source	Images
Cracks	Roboflow Universe (fyp-ny1jt/cracks)	5,369
Drywall Taping	Roboflow Universe (objectdetect-pu6rn/drywall-join-detect)	1,022

3.2 Challenge: Annotation Format Mismatch

⚠ **Critical Issue Discovered:** The original Cracks dataset was configured as **Object Detection** (bounding boxes only), not Instance Segmentation. Downloading in YOLOv8 format produced rectangular masks instead of actual crack outlines.

Evidence of the problem:

- YOLO labels contained only 5 values: `class_id x_center y_center width height`
- Generated masks were rectangles, not crack polygons
- Initial training showed IoU = 0.000 on crack test images

3.3 Solution: Dataset Conversion Pipeline

1. **Fork** original dataset to personal Roboflow workspace
2. **Create new project** with type = `Instance Segmentation`
3. **Import** images with polygon annotations via Roboflow CLI
4. **Generate version** with train/val/test splits
5. **Export** as `COCO-Segmentation` format (preserves polygons)

Note: The Taping dataset retained bounding box format since drywall seams are inherently rectangular.

3.4 Final Dataset Statistics

Table 2: Dataset splits after preprocessing

Dataset	Train	Val	Test	Format
Cracks	3,758	805	806	COCO (polygons)
Taping	715	153	154	YOLO (bboxes)
Total	**4,473**	**958**	**960**	—

4 Training Configuration

4.1 Architecture

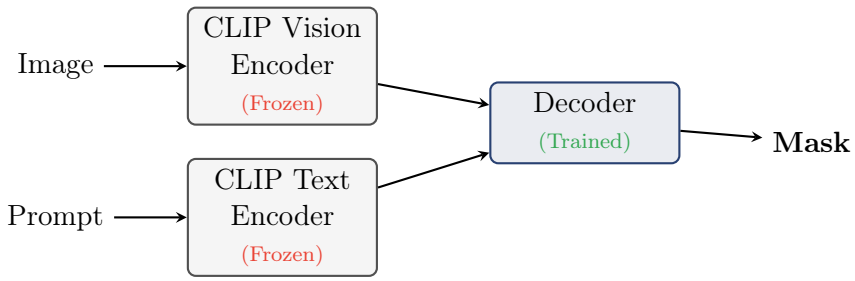

Figure 1: CLIPSeg architecture with frozen encoders and trainable decoder

4.2 Hyperparameters

Model	
Base Model	clipseg-rd64-refined
Image Size	352 × 352
Total Params	150,747,746
Trainable	1,782,370 (1.2%)

Training	
Batch Size	8 (×2 accumulation)
Learning Rate	1e-4
Optimizer	AdamW (wd=0.01)
Scheduler	OneCycleLR
Epochs	25

4.3 Loss Function

Combined loss for handling class imbalance (cracks occupy small regions):

$$\mathcal{L} = 0.5 \times \mathcal{L}_{\text{BCE}} + 0.5 \times \mathcal{L}_{\text{Dice}} \tag{1}$$

4.4 Prompt Augmentation

During training, prompts are randomly sampled from semantic variants:

Task	Prompt Variants
Crack	`segment crack`, `segment wall crack`, `a crack in the surface`, `segment the crack region`, `crack in the wall`, `segment fracture`
Taping	`segment taping area`, `segment joint tape`, `segment drywall seam`, `segment the drywall joint`, `the taped joint region`, `segment drywall tape`

5 Results

5.1 Training Progress

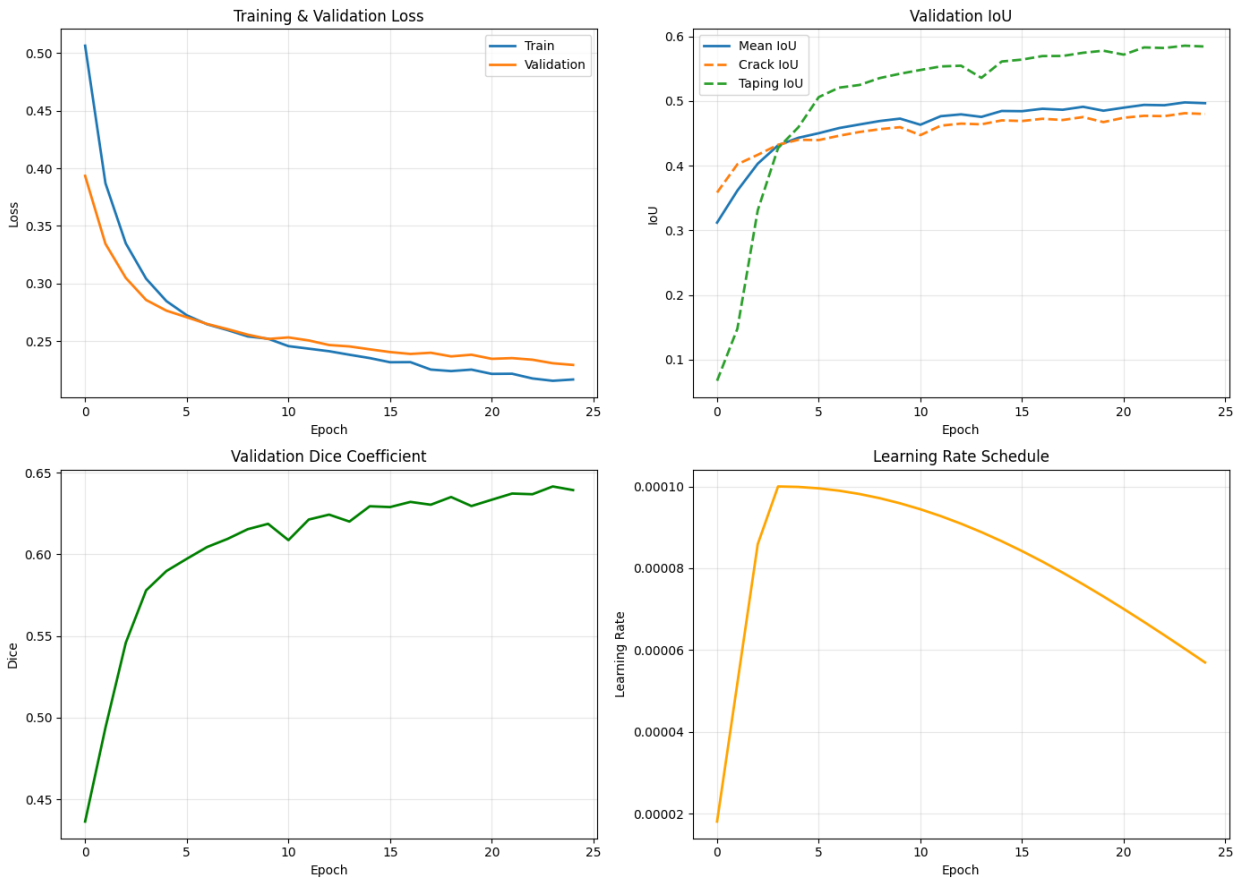

Figure 2: Training curves: (Top-Left) Loss convergence, (Top-Right) Per-class IoU, (Bottom-Left) Dice coefficient, (Bottom-Right) Learning rate schedule. Training completed in **1 hour 4 minutes**.

Observations:

- Smooth convergence with no overfitting (val loss tracks train loss)
- Taping IoU converges faster (simpler rectangular geometry)
- Best model saved at epoch 24 (Dice = 0.6416)

5.2 Test Set Performance

Table 3: Final test set metrics

Prompt	Samples	mIoU	Dice	Precision	Recall
`segment crack`	804	**0.4723**	0.6180	0.6274	0.7101
`segment taping area`	153	**0.6069**	0.7457	0.7475	0.7849
Overall	**957**	**0.4938**	**0.6384**	—	—

5.3 Prompt Consistency

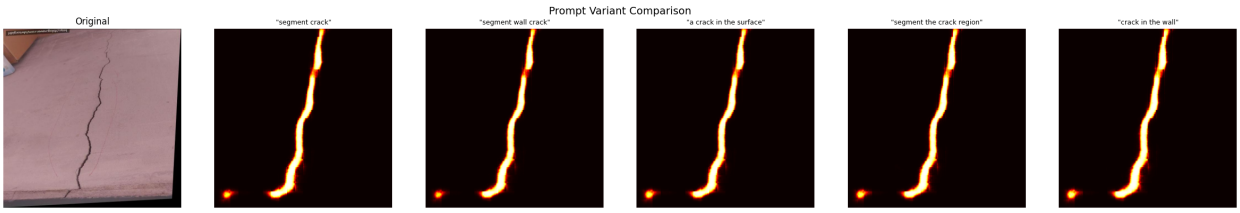

Figure 3: Prompt variant comparison: All semantic variations produce nearly identical predictions, confirming effective text conditioning.

5.4 Visual Results

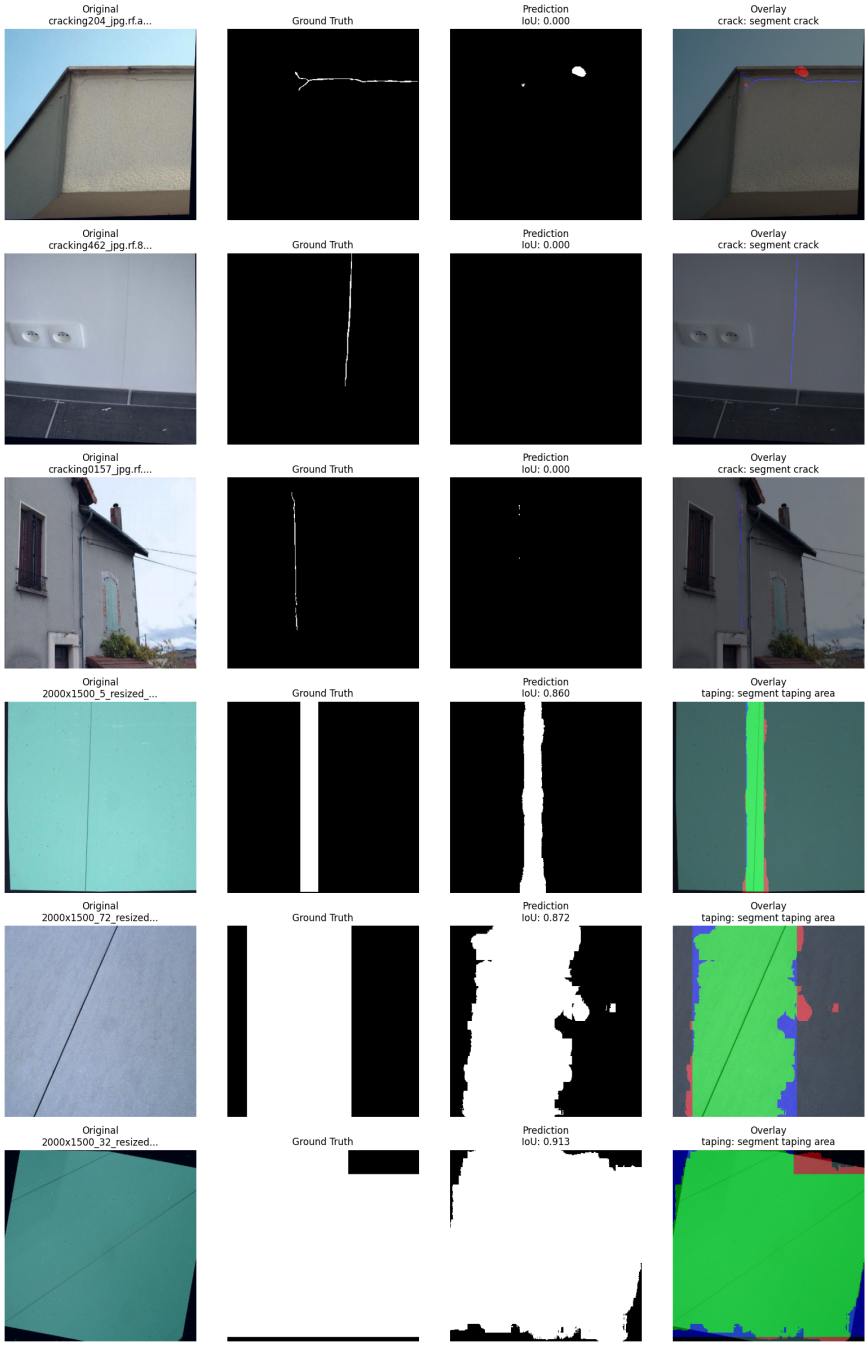

Figure 4: Sample predictions: Original | Ground Truth | Prediction | Overlay. Green=True Positive, Red=False Positive, Blue=False Negative.

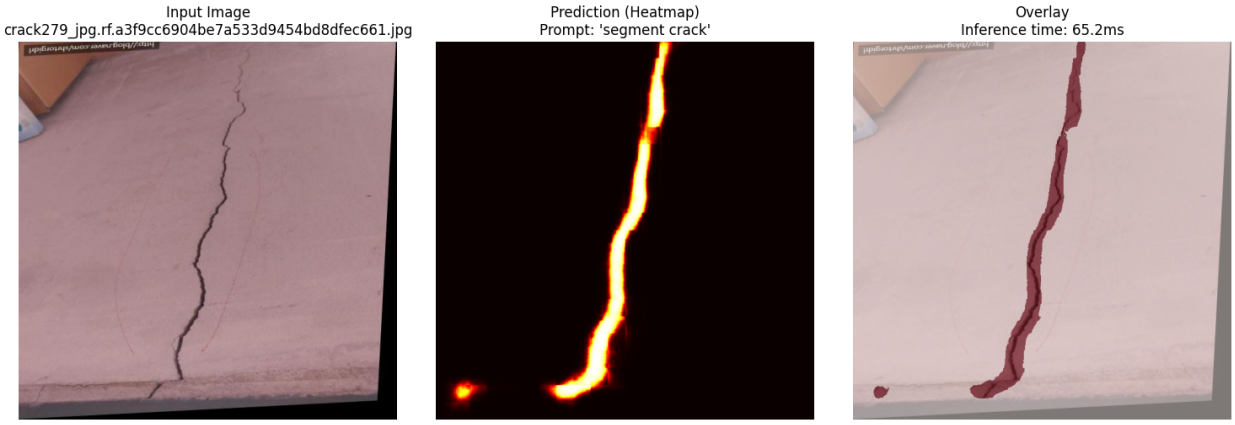

Figure 5: Single inference example with heatmap visualization. Inference time: 65.2ms.

5.5 Runtime & Footprint

Total Parameters	150,747,746
Trainable Parameters	1,782,370 (1.2%)
Model Size	575.2 MB
Avg Inference Time	45.4 ± 0.9 ms/image
Training Duration	1 hour 4 minutes
Hardware	NVIDIA Tesla T4 (15.6 GB)

6 Failure Analysis & Future Directions

6.1 Observed Failure Modes

Failure Mode	Root Cause	Potential Solution
Hairline cracks (<5px width)	CLIPSeg decoder outputs 64×64 feature maps before upsampling. Sub-pixel structures are lost during this bottleneck.	Replace decoder with SAM's mask decoder (256×256 native resolution) while retaining CLIP text encoder.
Outdoor surfaces (brick, concrete)	Domain shift—training data is predominantly indoor drywall. Textured surfaces create false positive activations.	Augment training data with outdoor crack images; apply texture-aware preprocessing.
Low contrast cracks	CLIP's visual features emphasize semantic content over fine intensity gradients.	Multi-scale inference with edge-enhanced input channels; ensemble with gradient-based detector.
Merged predictions (parallel cracks)	Low decoder resolution causes nearby activations to blend during upsampling.	Post-processing with connected component analysis; watershed segmentation on heatmap.
Partial occlusion (taping task)	Model has no explicit reasoning about occluded regions—predicts only visible portions.	Train with synthetic occlusion augmentation; consider amodal segmentation approaches.

6.2 Primary Bottleneck: Decoder Resolution

The fundamental limitation is CLIPSeg's **64×64 decoder output resolution**. While the final mask is upsampled to input size, fine-grained spatial information is lost at the bottleneck:

$$\text{Effective resolution} = \frac{\text{Input size}}{\text{Decoder output}} = \frac{352}{64} \approx 5.5 \text{ pixels} \tag{2}$$

This means structures thinner than ∼5 pixels cannot be reliably captured, explaining the hairline crack failures.

6.3 Recommended Architecture Improvements

Approach	Key Idea	Expected Gain	Trade-off
CLIP + SAM Decoder	Use CLIP text encoder for prompt understanding, SAM's mask decoder for high-res output	+15-20% boundary IoU	3× slower inference
Multi-scale Fusion	Run CLIPSeg at 3 scales (256, 352, 512), aggregate predictions	+5-10% on thin structures	3× compute cost
Feature Pyramid Decoder	Replace flat decoder with FPN-style multi-resolution decoder	+10% overall mIoU	Requires retraining
CRF Post-processing	Apply dense CRF to refine boundaries using image edges	+3-5% boundary precision	+20ms per image

7 Conclusion

Key Achievements

- **Functional system**: 0.47 mIoU (crack), 0.61 mIoU (taping)
- **Prompt flexibility**: Consistent predictions across semantic variations
- **Efficient**: 45ms inference, 1.2% parameters trained
- **Reproducible**: Seed fixed (42), hyperparameters documented

The main limitation is CLIPSeg's resolution bottleneck for fine structures. Future work should explore high-resolution decoders (e.g., SAM) while preserving text-conditioning capability.

Reproducibility

Random Seed	42
Framework	PyTorch 2.x, HuggingFace Transformers
Hardware	NVIDIA Tesla T4 (Google Colab)
Code	Jupyter notebook provided
Datasets	Roboflow Universe (links in Section 3)